

TASK 5 — CAPSTONE PROJECT & INCIDENT RESPONSE

Web Application Penetration Testing & Incident Response Simulation
DVWA-Based Security Assessment

Final Capstone Project Report

Cybersecurity & Ethical Hacking Internship
ApexPlanet

Name	K R Sri Krishna Tulasi
ID	APSPL2643218
Course / Program	Cybersecurity & Ethical Hacking Internship Program
Date	19 September 2026
Task	Task – 5

2. Executive Summary

This capstone project presents a controlled web application security assessment and incident response simulation using Damn Vulnerable Web Application (DVWA) in a local laboratory environment. The assessment combines reconnaissance and scanning, controlled SQL Injection testing, evidence collection, root-cause analysis, remediation, and recovery verification.

The assessment demonstrated SQL Injection against the vulnerable DVWA configuration and showed its potential to expose database records. Apache access logs were then reviewed to identify corresponding suspicious requests. The vulnerable implementation was traced to direct use of user-controlled input in an SQL query. The protected implementation uses a prepared statement with parameter binding, and the same SQL Injection test was checked under DVWA's Impossible security level.

The resulting workflow follows the incident-response sequence required for the capstone: detect the simulated attack, analyze evidence, contain the affected functionality, eradicate the underlying weakness, verify recovery, and document lessons learned.

3. Project Overview

The project follows the Web Application Penetration Testing / DVWA capstone route. It uses a deliberately vulnerable application in a controlled local laboratory so reconnaissance, exploitation, logging, mitigation, and incident-response activities can be demonstrated without targeting external systems.

The supplied evidence package contains DVWA screenshots, Nmap reconnaissance/scanning output, SQL Injection results, Apache access-log evidence, vulnerable and protected source-code views, and supporting web-application security findings.

4. Objectives

- Establish and document a controlled web-application security testing environment.
- Perform reconnaissance and service enumeration against the laboratory target.
- Assess representative DVWA web vulnerabilities.
- Demonstrate SQL Injection and document its impact.
- Correlate application testing with Apache access logs for incident detection.
- Analyze the vulnerable implementation and identify the root cause.
- Verify secure implementation using prepared/parameterized SQL.
- Simulate containment, eradication, recovery, and verification.
- Produce a professional capstone report with findings, evidence, mitigations, and lessons learned.

5. Scope

Testing is limited to the local DVWA laboratory environment and the associated Kali Linux assessment workstation. The target used in the supplied scan evidence is 192.168.56.104.

In scope: reconnaissance, scanning, DVWA configuration review, SQL Injection testing, supporting XSS/CSRF observations, source-code review, Apache log analysis, remediation verification, and incident-response documentation.

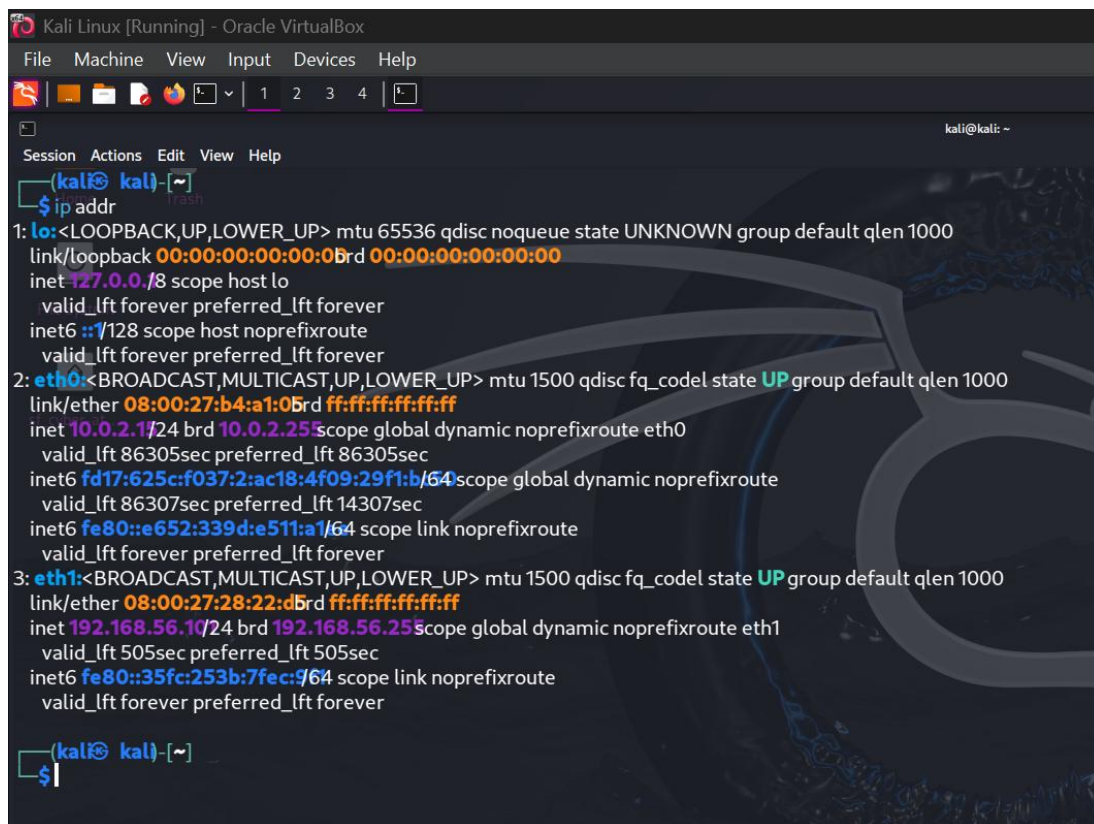
Out of scope: external systems, production applications, unrelated third-party infrastructure, and systems without authorization.

6. Tools & Technologies

- Kali Linux
- DVWA
- Apache HTTP Server
- MySQL
- Nmap
- Firefox/web browser
- Linux command-line tools (grep, tail, find, sed)
- PHP/PDO
- Prepared statements and parameter binding

7. Capstone Environment / Lab Setup

The supplied evidence shows a Kali Linux assessment environment and a DVWA web-application environment hosted in a local VirtualBox-based laboratory. The target address used during scanning is 192.168.56.104.



```
Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Session Actions Edit View Help
(kali@kali)~$ ip addr
1: lo:<LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0:<BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:b4:a1:05 brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 86305sec preferred_lft 86305sec
   inet6 fd17:625c:f037:2:ac18:4f09:29f1:b/64 scope global dynamic noprefixroute
       valid_lft 86307sec preferred_lft 14307sec
   inet6 fe80::e652:339d:e511:a1/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1:<BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:28:22:d5 brd ff:ff:ff:ff:ff:ff
   inet 192.168.56.104/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
       valid_lft 505sec preferred_lft 505sec
   inet6 fe80::35fc:253b:7fec:96/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
(kali@kali)~$
```

Figure 1. Kali Linux and DVWA laboratory evidence.




Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to **practice some of the most common web vulnerabilities**, with various levels of difficulty, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any internet facing servers**, as they will be compromised. It is recommend using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can download and install [XAMPP](#) for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

More Training Resources

DVWA aims to cover the most commonly seen vulnerabilities found in today's web applications. However there are plenty of other issues with web applications. Should you wish to explore any additional attack vectors, or want more difficult challenges, you may wish to look into the following other projects:

- [Mutillidae](#)
- [OWASP Vulnerable Web Applications Directory](#)

You have logged in as 'admin'

Username: admin
Security Level: Security Level: impossible
Locale: en
SQLi DB: mysql

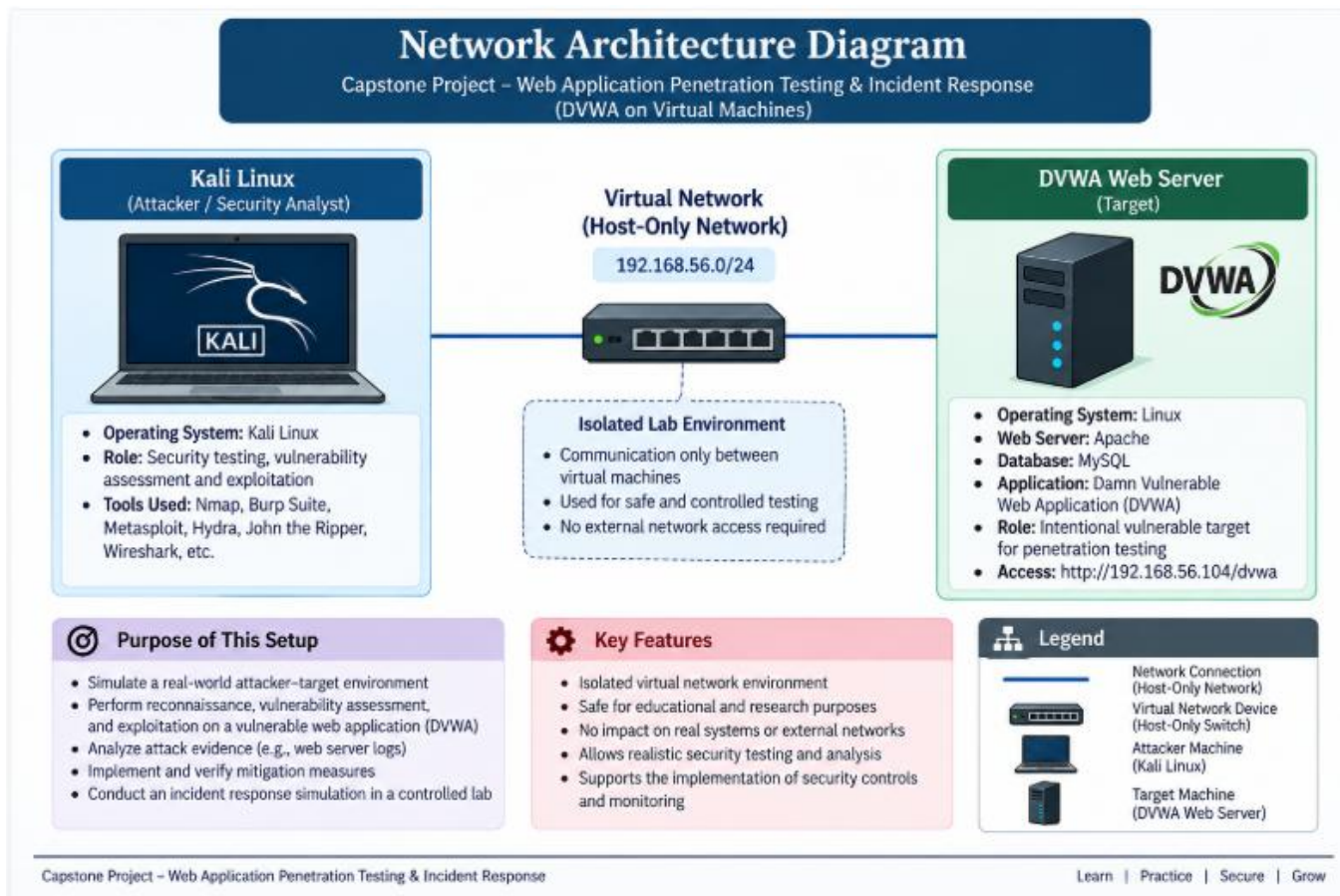
Damn Vulnerable Web Application (DVWA)

Figure 2. VirtualBox network configuration evidence.

The environment is intended for isolated, controlled security testing.

8. Network / Architecture Diagram

The final network/architecture diagram will be inserted in this section during the final revision. It will show Kali Linux, the isolated laboratory network, the DVWA web server, and the database/backend components.



9. Methodology

- Reconnaissance — identify the laboratory target and available services.
- Scanning and enumeration — identify exposed services and gather service/OS information.
- Vulnerability assessment — review the DVWA attack surface.
- Controlled exploitation — demonstrate SQL Injection in the intentionally vulnerable module.
- Evidence collection — preserve application, source-code, and server-log evidence.
- Incident analysis — correlate suspicious requests with application behavior and source code.
- Containment and eradication — use the protected configuration and parameterized SQL.
- Recovery and verification — repeat the controlled test and compare the outcome.
- Reporting — consolidate findings, mitigations, lessons, and evidence.

10. Reconnaissance & Scanning

Reconnaissance and scanning were performed within the isolated laboratory. The supplied Nmap evidence identifies 192.168.56.104 as an active host and shows multiple services exposed on the target.

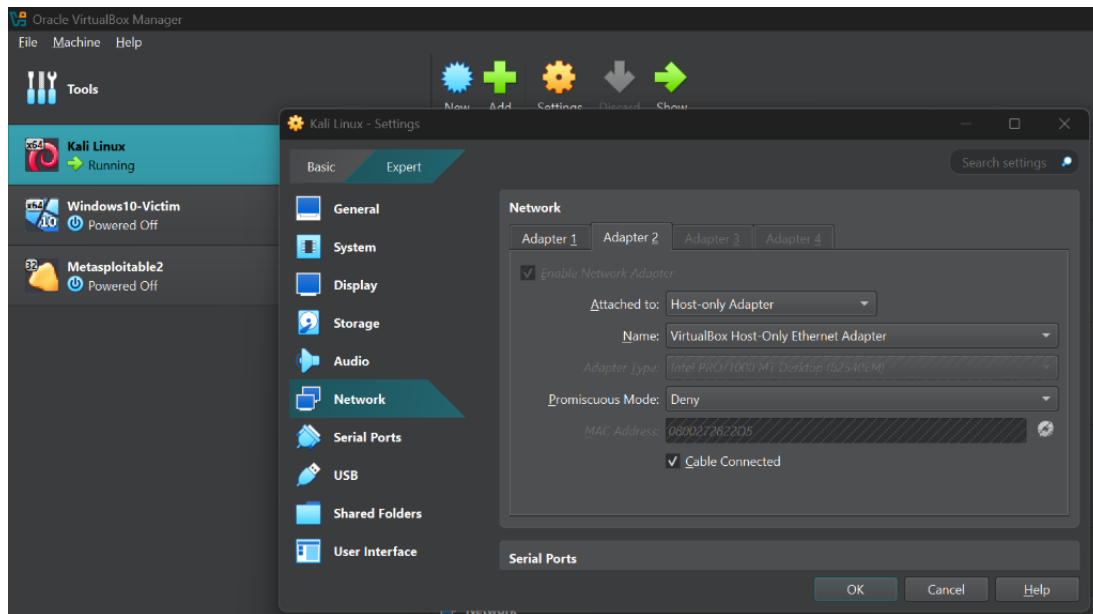


Figure 3. Nmap UDP/top-port scanning evidence for 192.168.56.104.

```

(kali)~$ nmap -sU --top-ports 20 192.168.56.104
Starting Nmap 7.98 ( https://nmap.org ) at 2026-08-15 17:59 +0530
Nmap scan report for 192.168.56.104
Host is up (0.0015s latency).

PORT      STATE      SERVICE
53/udp    open       domain
67/udp    closed     dhcp
68/udp    open|filtered dhcp
69/udp    open|filtered tftp
123/udp   closed     ntp
135/udp   closed     msrpc
137/udp   open       netbios-ns
138/udp   open|filtered netbios-dgm
139/udp   closed     netbios-ssn
161/udp   closed     snmp
162/udp   closed     snmptrap
445/udp   closed     microsoft-ds
500/udp   closed     isakmp
514/udp   closed     syslog
520/udp   closed     route
631/udp   closed     ipp
1434/udp  closed     ms-sql-m
1900/udp  closed     upnp
4500/udp  closed     nat-t-ike
49152/udp closed     unknown
MAC Address: 08:00:27:4B:AF:2C (Oracle VM VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 16.10 seconds

```

Figure 4. Nmap TCP service enumeration and OS-detection evidence.

The scan results show a broad service surface including FTP, SSH, Telnet, SMTP, HTTP, DNS, SMB-related services, and database-related services. These results provided reconnaissance context for the subsequent application-level assessment.

11. Vulnerability Assessment

DVWA was used to assess representative web-application weaknesses in a deliberately vulnerable training environment. SQL Injection is the primary capstone incident-response finding. Stored XSS and Reflected XSS are retained as supporting findings.



Figure 5. DVWA security-level configuration evidence.

12. Controlled Exploitation — SQL Injection

SQL Injection was demonstrated against the DVWA SQL Injection module in the vulnerable configuration. Crafted input altered the intended SQL query and produced unauthorized database output.

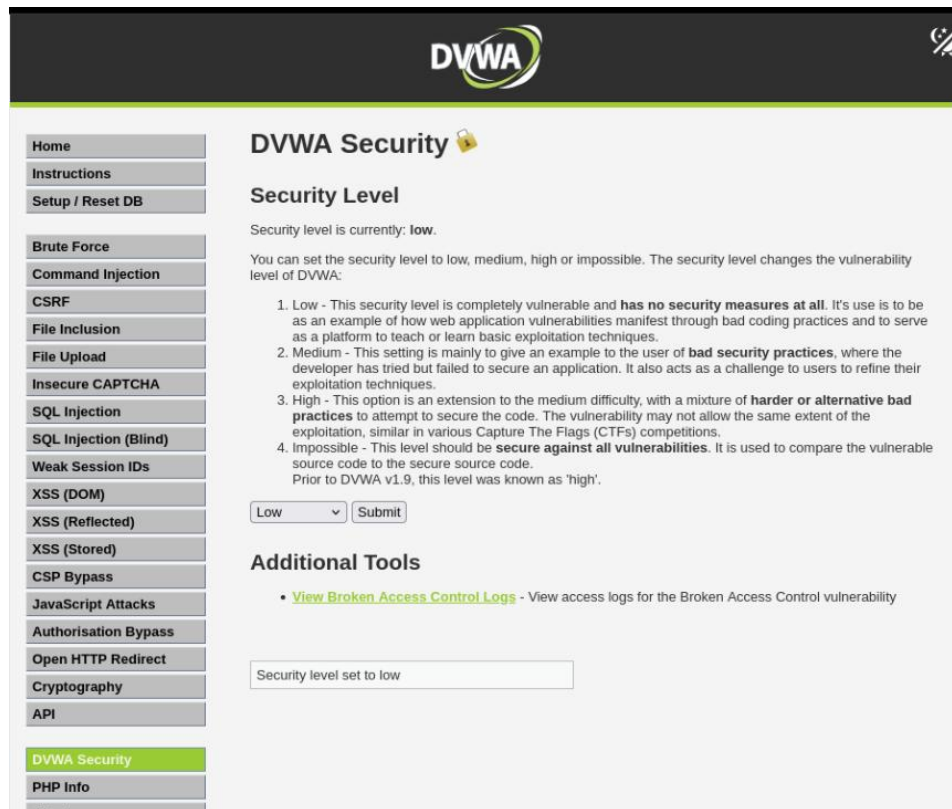


Figure 6. DVWA SQL Injection testing and vulnerable behavior.

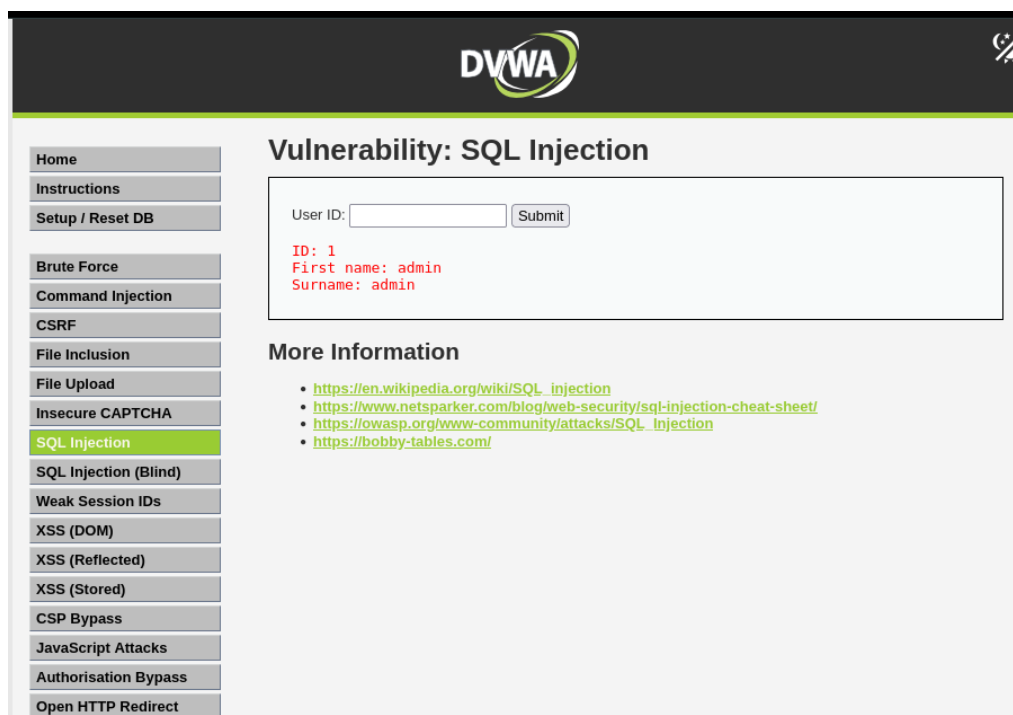


Figure 7. SQL Injection impact showing database records returned by the vulnerable application.

The demonstrated impact included retrieval of user-related database information, including password-hash values in the training environment.

13. Findings & Impact

13.1 Primary Finding — SQL Injection

The primary finding is SQL Injection in the DVWA SQL Injection functionality. The vulnerable application returned records beyond the intended lookup, demonstrating database query manipulation.

The principal security impact demonstrated is a confidentiality risk because crafted application input could be used to retrieve database information not intended to be exposed by the user-ID lookup.

13.2 Supporting Findings

Stored XSS and Reflected XSS were also observed in the broader DVWA assessment. They are retained as supporting findings rather than separate incident-response cases.

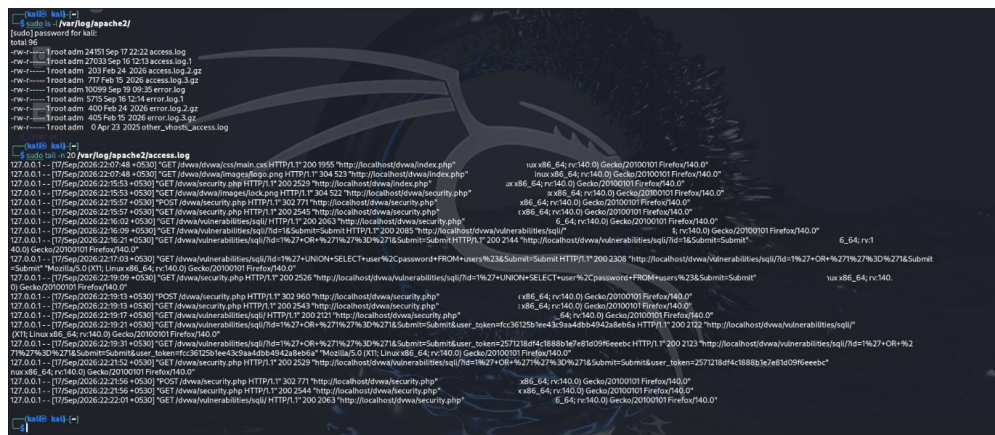


Figure 8. Supporting Stored XSS evidence.



Figure 9. Supporting Reflected XSS evidence.

14. Root Cause Analysis

The root cause of the SQL Injection finding was direct use of user-controlled input in SQL query construction in the vulnerable implementation.

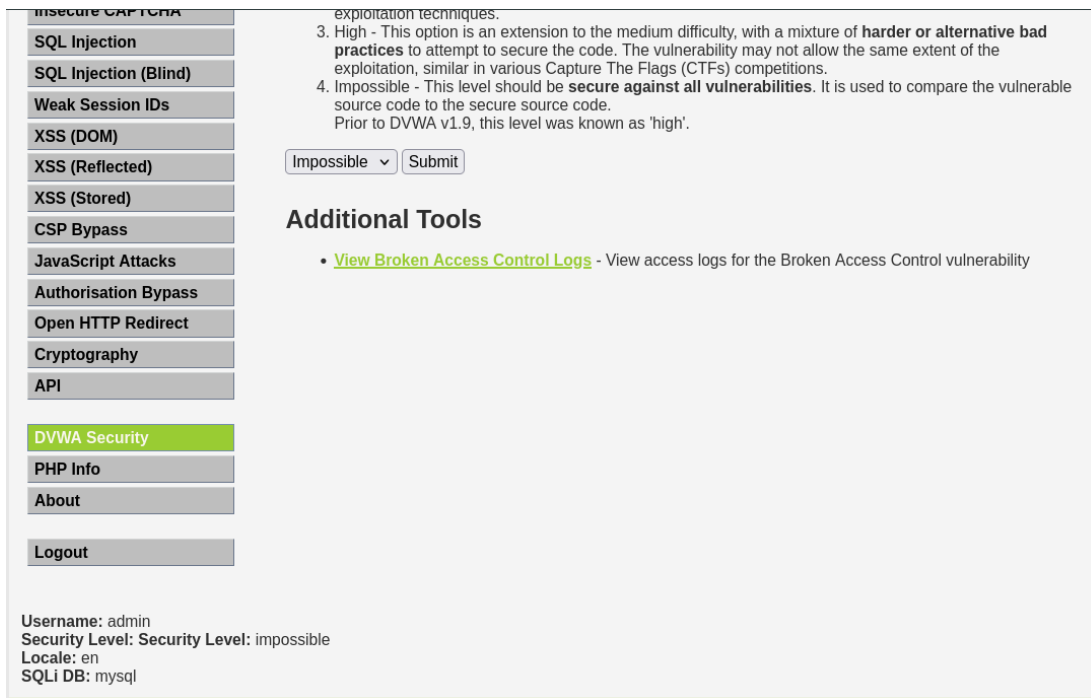


Figure 10. Vulnerable low.php implementation showing direct incorporation of the user-controlled identifier into the SQL query.

The vulnerable implementation obtains the identifier from the request and constructs a query around that value. Because the input is incorporated into the SQL statement rather than being treated strictly as data, crafted input can change the query structure.

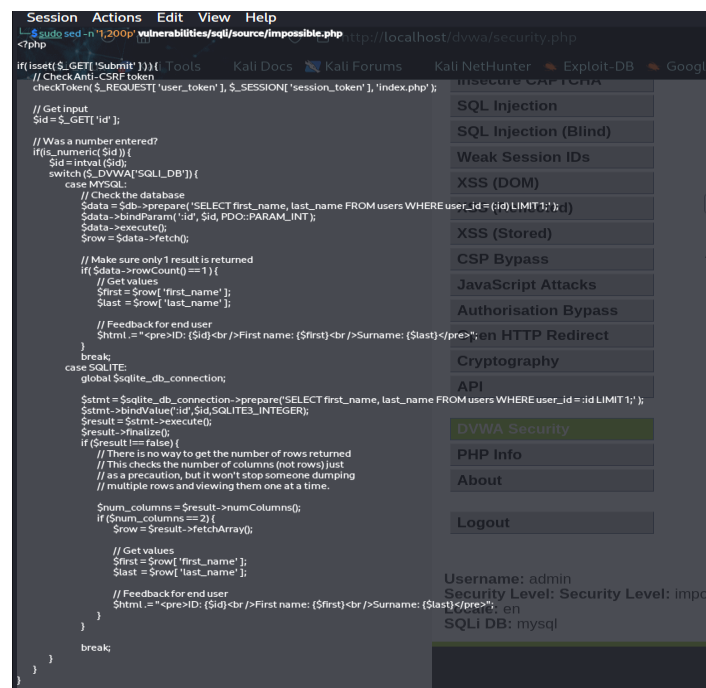


Figure 11. Protected implementation showing prepared statement and parameter binding.

The protected implementation separates SQL structure from user-supplied data by preparing the query and binding the identifier as a parameter.

15. Incident Response Simulation

The SQL Injection activity is treated as the simulated incident. The response is organized into detection, analysis, containment, eradication, recovery, and verification.

15.1 Detection

Apache access logs were reviewed to identify the server-side record of the SQL Injection activity. The supplied logs contain requests to the DVWA SQL Injection endpoint from the local source address 127.0.0.1.

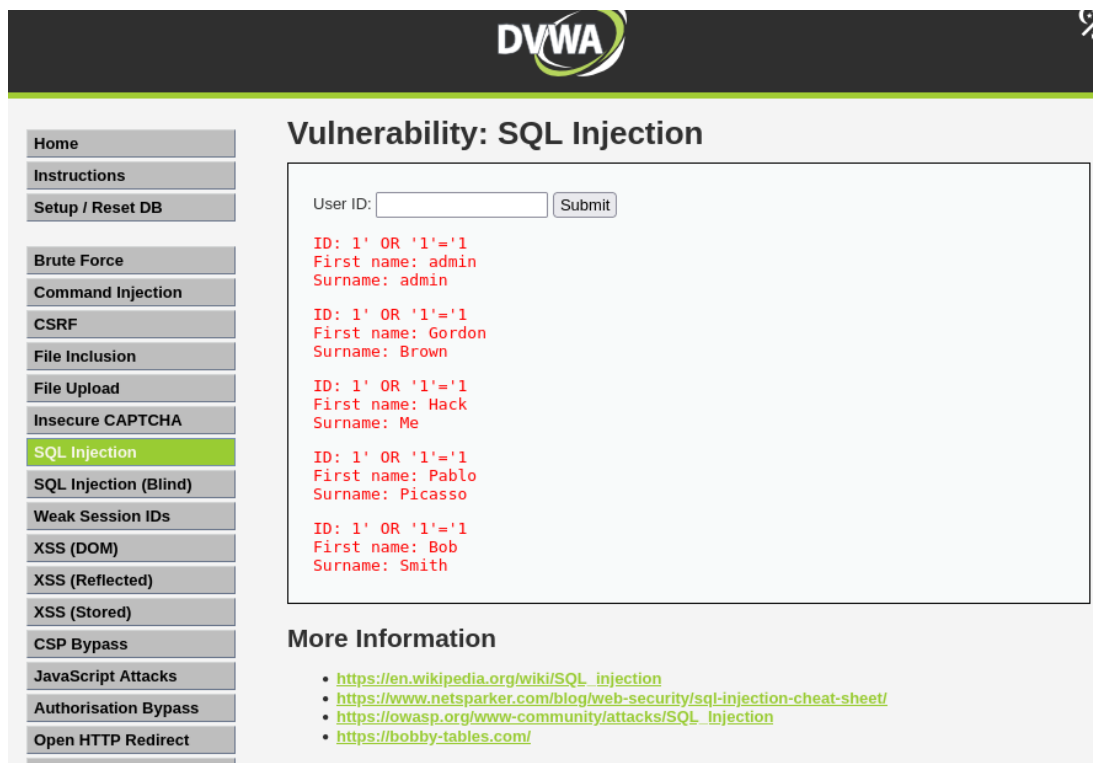


Figure 12. Apache access-log entries filtered for the DVWA SQL Injection endpoint.

A second log query isolates requests containing the UNION indicator. The recorded request includes an encoded representation of a UNION SELECT query targeting user/password fields.

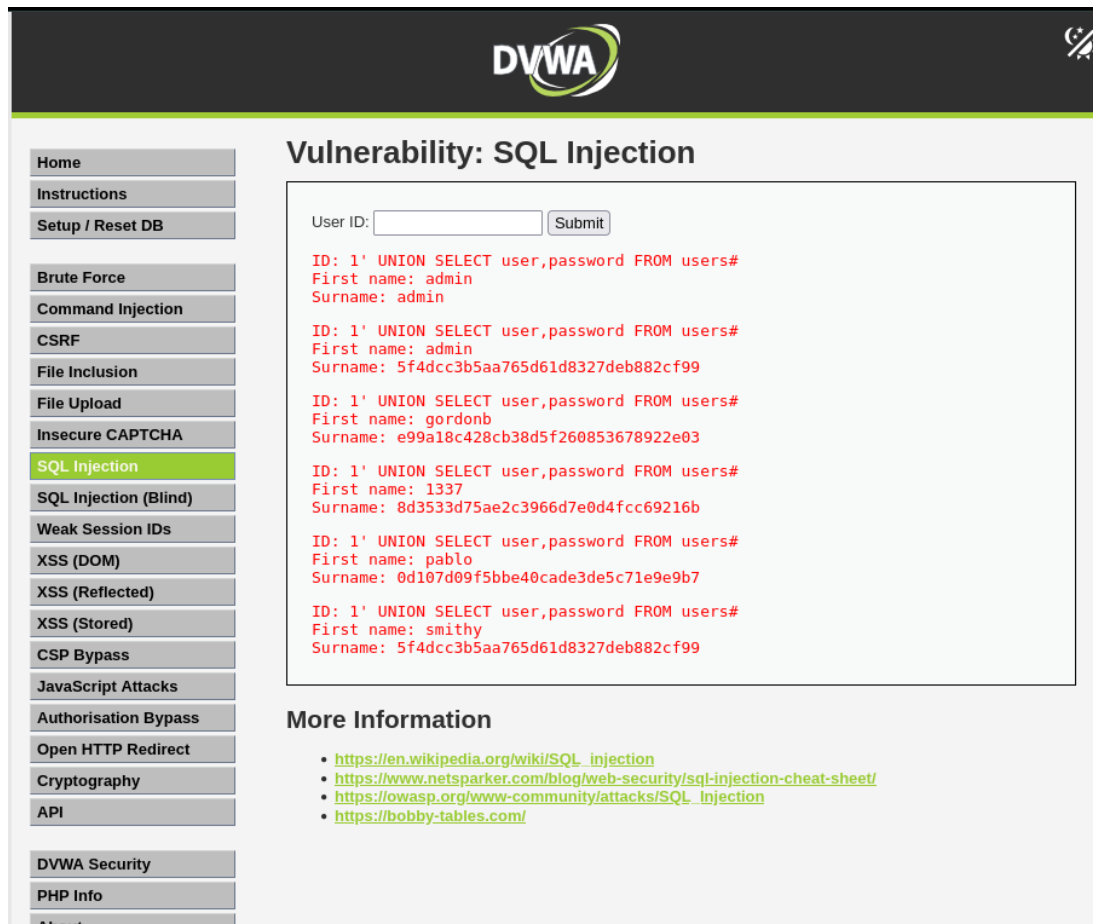


Figure 13. Apache log evidence isolating the UNION SQL Injection indicator.

15.2 Analysis

The analyst correlated the DVWA SQL Injection result, Apache access-log evidence, and vulnerable source code. The log establishes that the suspicious HTTP request reached the server; the application evidence establishes the observed SQL Injection behavior; and the source-code evidence identifies the direct SQL construction responsible for the weakness.

Indicator	Observed evidence
Source	127.0.0.1 in the local laboratory
Target endpoint	/dvwa/vulnerabilities/sqli/
HTTP method	GET in the recorded requests
Suspicious input	OR / UNION-based SQL Injection indicators
SQLi indicator	UNION SELECT user,password FROM users
HTTP result	HTTP 200 recorded for the demonstrated requests
Affected component	DVWA SQL Injection module

15.3 Containment

Containment was represented in the laboratory by moving the DVWA testing state to the protected Impossible security level rather than continuing to operate the vulnerable configuration. This establishes a protected state for recovery testing.

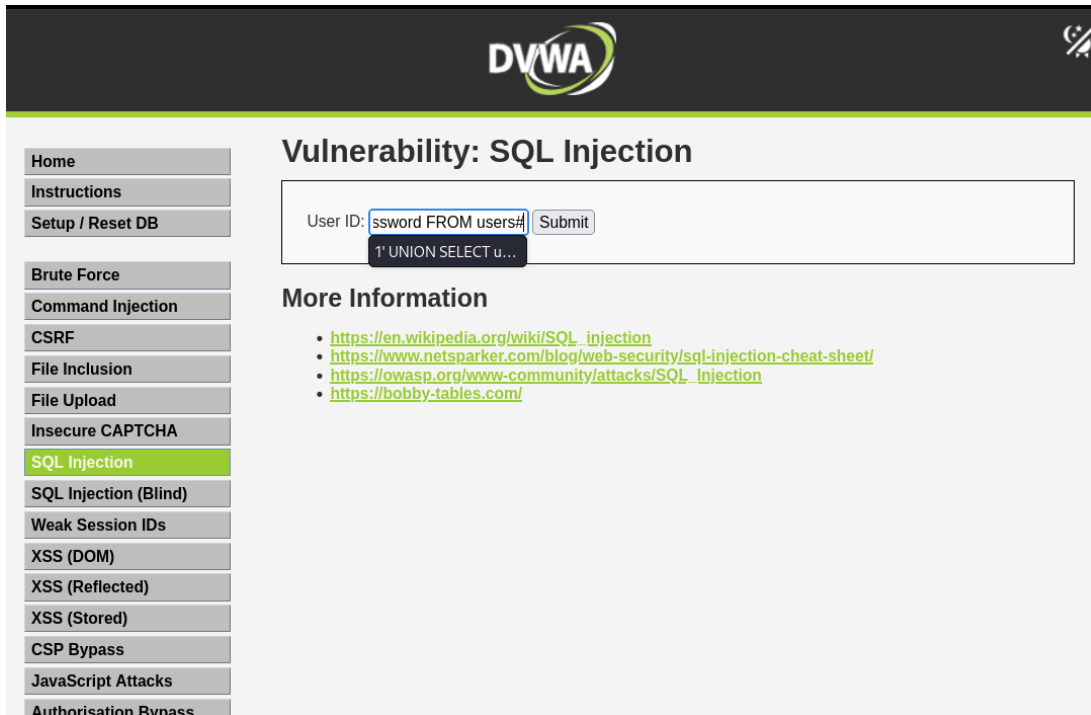


Figure 14. DVWA configured at the Impossible security level.

15.4 Eradication

The underlying weakness was addressed through the protected implementation using a prepared SQL statement and parameter binding. This removes the vulnerable pattern of constructing the query directly from user-controlled SQL text.

The protected implementation shown in Figure 11 uses a prepared statement, binds the identifier as an integer parameter, and executes the prepared query.

15.5 Recovery

After the protected state was established, the previously used SQL Injection test input was submitted again. The earlier unauthorized database output was not reproduced under the protected configuration.

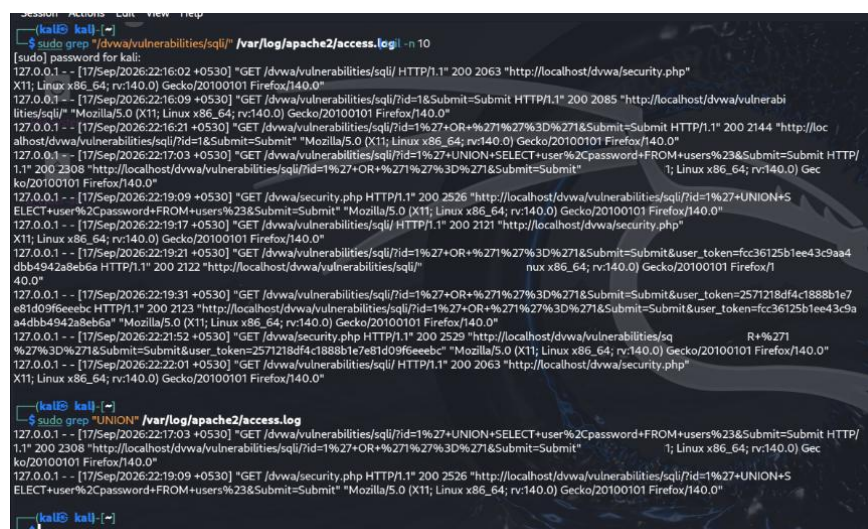


Figure 15. Recovery verification: the previously used SQL Injection test input shown under the protected configuration.

15.6 Verification

Verification compares the vulnerable-state evidence with the protected-state evidence. The vulnerable configuration demonstrated query manipulation and unauthorized output, while the protected implementation uses parameterized SQL and the repeated test did not reproduce the earlier database disclosure.

This is a controlled laboratory validation; a single test does not prove that every possible SQL Injection variant has been eliminated.

16. Mitigation & Security Recommendations

Recommendation	Purpose
Prepared/parameterized SQL	Separate SQL code from user-controlled data.
Server-side input validation	Enforce expected data types and reject unexpected input.
Database least privilege	Limit application database permissions to required operations.
CSRF protection	Use and validate anti-CSRF tokens for state-changing requests.
Output encoding	Encode untrusted output to reduce XSS risk.
Security headers	Use appropriate headers such as Content-Security-Policy as defense in depth.
Log monitoring	Review and correlate web-server/application logs for suspicious requests.
Post-remediation retesting	Repeat the original security test and preserve evidence of the result.
Lab isolation	Keep deliberately vulnerable applications on private/host-only networks.

17. Post-Incident Analysis / Lessons Learned

- Application testing and server-side logging provide complementary evidence.
- Preserving timestamps, source information, endpoints, and request indicators improves incident reconstruction.
- Source-code review connects an observed attack to its underlying programming pattern.
- Prepared statements directly address the demonstrated SQL Injection root cause.
- Recovery testing should reproduce the original condition as closely as practical and compare before/after results.
- Deliberately vulnerable applications should remain isolated from production or public networks.
- Supporting findings such as XSS and CSRF should be tracked even when SQL Injection is the primary incident-response case.

18. Conclusion

The Task 5 capstone combines web-application penetration testing with a controlled incident-response simulation. SQL Injection was demonstrated in a deliberately vulnerable DVWA configuration, its impact was documented, corresponding Apache log activity was identified, the vulnerable source was analyzed, and a protected implementation based on prepared statements and parameter binding was verified.

The project therefore covers the capstone flow from reconnaissance and controlled exploitation through detection, analysis, containment, eradication, recovery, and verification. The final revision requires only insertion of the finalized network/architecture diagram and final submission links.

19. Evidence Index

ID	Evidence	Purpose
E01	Kali Linux / DVWA laboratory	Environment
E02	VirtualBox network configuration	Environment
E03	Nmap UDP/top-port scan	Reconnaissance
E04	Nmap TCP service enumeration / OS detection	Scanning
E05	DVWA security configuration	Assessment
E06	SQL Injection vulnerable behavior	Exploitation
E07	SQL Injection database-record impact	Impact
E08	Stored XSS evidence	Supporting finding
E09	Reflected XSS evidence	Supporting finding
E10	Vulnerable low.php	Root cause
E11	Protected prepared-statement implementation	Eradication
E12	Apache log — SQLi requests	Detection
E13	Apache log — UNION indicator	Detection
E14	DVWA Impossible security level	Containment
E15	SQL Injection recovery verification	Recovery / verification

20. References

- ApexPlanet Internship Task 5 brief — Capstone Project & Incident Response.
- Damn Vulnerable Web Application (DVWA) — local training application used in the controlled assessment.
- Nmap output generated during the authorized laboratory scan.
- Apache HTTP Server access logs generated by the local DVWA environment.
- Protected PHP/PDO implementation used for SQL Injection mitigation.